Layer 2: Stochastic MIP + Pareto Frontier

Document Owner: Zernitsky Itamar **Status:** Design Reference — Architecture Planning **Scope:** Complete technical specification of the stochastic MIP: problem formulation, all constraints and objectives, linearization strategies, SAA chance constraints, ε-constraint Pareto frontier, solver configuration, debugging, and validation.

> **What this layer does.** Layer 1 produces 200 plausible demand futures. Layer 2 decides — simultaneously and optimally — which blocks to assign to which providers, such that every hard operational constraint is respected and the allocation is robust across all 200 futures. It does this four times, once per Pareto theme, producing a frontier of non-dominated candidates for the committee to choose from.

> **Consistent example throughout.** All illustrations use SITE-001: 18 providers, 4 OR rooms, T*=2 rotation, 13 historical weeks, 411 blocks, S=200 scenarios, 3 goals (Garcia target 70%, Thompson target 70%, Wright add 16 hrs).

Table of Contents

    - Linearization Strategies
- Stage 4 — Pareto Frontier via ε-constraint
- Stage 5 — Solution Extraction
- Stage 6 — Pareto Candidate Selection and Recommendation
- Solver Configuration and Debugging
- Layer 2 Artifact Bundle

Stage 0: Data Loading and Eligibility

What It Does

Before building the MIP, determine exactly which blocks and providers participate in the optimization. This step has significant impact on solve time — every ineligible variable you fix before solving reduces the problem size.

Eligibility Sets

ProvidersInOpt is the user-defined set of providers whose blocks are variable. Providers not in this set are **frozen** — their blocks are locked to the warm-start template assignment and cannot be moved.

```
╔══════════════════════════════════════════════════════════════════════════╗
║  ELIGIBILITY SETUP  ·  SITE-001  ·  411 blocks  ·  18 providers          ║
╠══════════════════════════════════════════════════════════════════════════╣
║                                                                          ║
║  ALL 18 providers included in ProvidersInOpt by default.                 ║
║                                                                          ║
║  GOAL CLASSIFICATION                                                     ║
║  ──────────────────────────────────────────────────────────────────────  ║
║  P_util  (target utilization goals → get SAA chance constraints):        ║
║    • PRV-003  Dr. Garcia    Cardiac   target 70%                         ║
║    • PRV-015  Dr. Thompson  Ortho     target 70%                         ║
║                                                                          ║
║  P_add   (add-hours goals → get hard floor constraints):                 ║
║    • PRV-006  Dr. Wright    Cardiac   add 16 hrs                         ║
║                                                                          ║
║  P_free  (no goal → blocks can move but no explicit coverage target):    ║
║    • All 15 remaining providers                                          ║
```

```
╔═══════════════════════════════════════════════════════════════════════════╗
║                                                                           ║
║  BLOCK ELIGIBILITY                                                        ║
║  ─────────────────────────────────────────────────────────────────────    ║
║  Total template blocks          :  411                                    ║
║  Pre-closed (status=CLOSED)     :    8   → C1: Σ_p assigned[p][b] = 0     ║
║  Frozen (non-included provider) :    0   → all 18 are included            ║
║  Flipped rooms (overlap pairs)  :    3   → C3: frozen to warm-start       ║
║  Eligible for optimization      :  400                                    ║
║                                                                           ║
║  MIP SIZE BEFORE ELIGIBILITY REDUCTION                                    ║
║  ─────────────────────────────────────────────────────────────────────    ║
║  assigned[p][b] vars  =  18 × 411 = 7,398  (most pre-fixed by constraints)║
║  z[p][s] vars         =   2 × 200 =   400  (for P_util only)              ║
║  slack[p] vars        =   2              (for P_util)                     ║
║  undershoot[p] vars   =   1              (for P_add)                      ║
║                                                                           ║
║  MIP SIZE AFTER COLUMN GENERATION (pre-fixing ineligible pairs)           ║
║  ─────────────────────────────────────────────────────────────────────    ║
║  Site exclusivity kills ~3,400 (p,b) pairs  (wrong site)                  ║
║  Room compatibility kills ~1,800 (p,b) pairs  (wrong room type)           ║
║  Active assigned vars  ≈  1,236   (tractable)                             ║
║                                                                           ║
╚═══════════════════════════════════════════════════════════════════════════╝
```

Warm-Start Matrix

The warm-start $W[p][b]$ is built from the Pre-Layer block template:

$$W[p][b] = 1 \quad \text{if } p = \text{dominant\_provider}(b), \quad 0 \text{ otherwise}$$

This is passed to the solver as an initial feasible solution. The solver starts from a near-optimal state rather than an empty assignment. On SITE-001 this cuts solve time by roughly 40% on the hardest theme (continuity_first).

Stage 1: Series Detection

What a Series Is

A series is a recurring block slot across the optimization horizon. On SITE-001 with T*=2 and 13-week quarter, a series for (OR-101, Tuesday, 07:00) has 13 instances — one per week. The template repeats every 2 weeks so the instances alternate between phase 0 and phase 1.

```
╔══════════════════════════════════════════════════════════════════════════╗
║  SERIES DETECTION  ·  SITE-001  ·  411 blocks → 32 series                ║
╚══════════════════════════════════════════════════════════════════════════╝

  SERIES DEFINITION:
  All block instances sharing (room_id, day_of_week, start_time)
  across all T* × n_weeks instances in the optimization period.

  Example — series for OR-101 Tuesday 07:00:
  BLK-047 (Phase 0, Week 1),  BLK-047b (Phase 1, Week 2),
  BLK-047c (Phase 0, Week 3), ...  BLK-047m (Phase 0, Week 13)
  → 13 instances, assigned to dominant provider Garcia (or Wright)

  ADJACENT PAIRS  (for fragmentation objective O2)
  ──────────────────────────────────────────────────────────────────────
  adj = consecutive (b1, b2) pairs ordered by date within each series.
  For a 13-instance series: 12 adjacent pairs.
  Total adj pairs across all 32 series: 32 × 12 = 384 pairs

  CONSISTENCY SCORES  (for objective O5)
  ──────────────────────────────────────────────────────────────────────
  consistency_score[p][series] =
    (instances in series where p is available) / (total instances)

  Available = not absent + within range_available + room compatible
              + site compatible

  Sample scores for OR-101 Tue series (13 instances):
    Garcia  → available 12/13 (1 absence in W09) → score = 0.92
    Wright  → available  7/13 (range_start = W07)  → score = 0.54
    Chen    → available 13/13                       → score = 1.00
    Torres  → available  0/13 (wrong room type)     → score = 0.00
```

Why Series Matter

Without series detection, the optimizer treats each block instance independently. It might give Garcia week-1 Tuesday, take it away week-2, give it back week-3 — a fragmented schedule the OR team would find operationally unworkable. Series detection provides the adjacency structure that makes fragmentation a computable objective.

Stage 2: BlockTimeRequired Computation

The Core Formula

For each scenario s and utilization-goal provider p, compute the quarterly block hours required to hit the target:

$$R[p, s] = \frac{Q^{(s)}_{\text{casetime}}[p] + Q^{(s)}_{\text{turnover}}[p]}{\text{TargetUtilization}[p]} + \text{EarlyRelease}_{\text{projected}}[p]$$

where the quarterly demand comes from scaling up the weekly scenarios from Layer 1:

$$Q^{(s)}_{\text{casetime}}[p] = \left(\sum_d \text{demand\_casetime}^{(s)}_{p,d}\right) \times n_{\text{weeks}}$$

$$Q^{(s)}_{\text{turnover}}[p] = \left(\sum_d \text{demand\_turnover}^{(s)}_{p,d}\right) \times n_{\text{weeks}}$$

And the deterministic early release projection:

$$\text{EarlyReleaseprojected}[p] = \alpha \cdot \sum_{b \in B^{\text{hist}}_p} \text{EarlyReleaseDur}_b$$

```
╔═══════════════════════════════════════════════════════════════════════════╗
║  R[p,s] MATRIX  ·  SITE-001  ·  P_util = {Garcia, Thompson}  ·  S=200      ║
╠═══════════════════════════════════════════════════════════════════════════╣
║                                                                           ║
║  DR. GARCIA  target=70%  EarlyRelease_projected=316 min = 5.27 hrs        ║
║  ─────────────────────────────────────────────────────────────────────    ║
║   s   |  Q_case (hrs) |  Q_turn (hrs) |  Q_total (hrs) |  R[Garcia,s]     ║
║  ─────┼────────────────┼────────────────┼────────────────┼──────────────    ║
║    1  |    17.6        |     2.9        |    20.5        |  20.5/0.70+5.27  ║
║       |                |                |                |  = 34.5 hrs      ║
║    2  |     9.8        |     1.6        |    11.4        |  21.6 hrs        ║
║    5  |    22.1        |     3.4        |    25.5        |  41.7 hrs        ║
║    6  |     0.0        |     0.0        |     0.0        |   5.3 hrs  ← ZI  ║
║       |                |                |    (zero wk)   |  (zero demand)   ║
║  ...  |                |                |                |                  ║
║                                                                           ║
║  R[Garcia,s] distribution over S=200:                                     ║
║    P10 = 23.1 hrs    P50 = 34.0 hrs    P90 = 44.5 hrs                     ║
║    Required coverage:  k = ⌈0.85 × 200⌉ = 170 scenarios                   ║
║                                                                           ║
║  DR. THOMPSON  target=70%  EarlyRelease_projected=198 min = 3.30 hrs      ║
║  ─────────────────────────────────────────────────────────────────────    ║
║  R[Thompson,s] distribution:                                              ║
║    P10 =  9.4 hrs    P50 = 12.6 hrs    P90 = 15.8 hrs                     ║
║                                                                           ║
║  KEY INSIGHT: R[Garcia,s] ranges from 5 hrs (zero-demand scenario)        ║
║  to 45 hrs (high-demand scenario). No single block count covers all       ║
║  200 scenarios. The optimizer finds the minimum feasible count that       ║
║  covers ≥170 scenarios — which is 2 blocks (16 hrs) since R≤16 in         ║
```

```
╟  roughly 2% of scenarios only. The binding factor is case_volume.            ╢
╟                                                                             ╢
╚═══════════════════════════════════════════════════════════════════════════╝
```

Why This Formula Matters

The old approach computes one number per provider (the α-scaled historical sum) and asks: "does the allocated time exceed the required time?" That is a deterministic check with a point estimate. This approach computes 200 numbers per provider — one per scenario — and asks: "in how many scenarios does the allocated time exceed the required time?" The fraction of scenarios covered is the formal probabilistic guarantee that replaces the vague "minimize deviation" objective of the existing engine.

Stage 3: MIP Construction

Decision Variables

The core assignment matrix:

$$\text{assigned}[p][b] \in 0,1 \quad \forall, p \in P_{\text{eligible}},\ b \in B_{\text{eligible}}$$

Scenario coverage indicators (only for utilization-goal providers):

$$z[p][s] \in 0,1 \quad \forall, p \in P_{\text{util}},\ s \in 1,\ldots,S$$

Infeasibility buffers (penalized near-hard):

$$\text{slack}[p] \geq 0 \quad \forall\, p \in P_{\text{util}}, \qquad \text{undershoot}[p] \geq 0 \quad \forall\, p \in P_{\text{add}}$$

On variable count and tractability. The full assignment matrix has $18 \times 411 = 7{,}398$ binary variables. After pre-fixing via site exclusivity, room compatibility, and C1–C3, only ≈1,236 remain active. The $z[p][s]$ matrix adds $2 \times 200 = 400$ binary variables. Total active binary variables: ≈1,636. This is tractable for CBC in under 60 seconds for all four themes.

Hard Constraints C1–C11

C1 — Pre-closed blocks

$$\sum_p \text{assigned}[p][b] = 0 \quad \forall\, b : \text{status}(b) = \text{PRE\_CLOSED}$$

Implementation note. Pre-fixed before solving — do not create assigned[p][b] variables for these blocks at all. Reduces model size at zero cost.

C2 — Warm-start freeze for non-included providers

$$\text{assigned}[p^*][b] = 1 \quad \forall\, b : \text{included}(p^*) = \text{False}$$

where $p^* = \text{warm\_start\_provider}(b)$. Mutual exclusivity (C4) ensures no other provider can be assigned the same block.

C3 — Flipped rooms freeze

A flipped room is a block that overlaps in time with another block sharing the same warm-start provider in the template — meaning the provider would need to be in two OR rooms simultaneously. These are artefacts of the template reconstruction (two consecutive blocks that overlap by a few minutes due to the ±2h clustering). Freeze them to warm-start to prevent the optimizer from "fixing" a bad template assignment.

$$\text{assigned}[p_b][b] = 1 \quad \forall\, b \in \mathcal{F}$$

$$\mathcal{F} = \{\, b \;\big|\; \exists\, b' \neq b : \text{ws}(b') = \text{ws}(b) \;\wedge\; b \in \mathcal{T} \;\wedge\; b' \in \mathcal{T} \;\wedge\; \text{overlap}(b, b') \}$$

C4 — Mutual exclusivity

$$\sum_p \text{assigned}[p][b] \leq 1 \quad \forall\, b \in B_{\text{eligible}}$$

Note: ≤1 not =1. A block may go to zero providers (become open time). It is not forced to be assigned to someone.

C5 — Site exclusivity

$$\text{assigned}[p][b] = 0 \quad \forall\, p,\ b : \text{site}(b) \notin \text{exclusive\_sites}(p)$$

Implementation note. Pre-fix these to zero and remove the variables entirely. This is the biggest variable reduction: on SITE-001 it eliminates ~3,400 (p,b) pairs.

C6 — Room compatibility

$$\text{assigned}[p][b] = 0 \quad \forall\, p, b : \text{room\_type}(b) \neq \text{room\_type\_required}(p)$$

Pre-fix to zero. Eliminates ~1,800 (p,b) pairs on SITE-001.

C7 — No time overlap

$$\text{assigned}[p][b_1] + \text{assigned}[p][b_2] \leq 1$$

$$\forall\, p,\ b_1 \neq b_2 \text{ s.t. } \text{end}(b_1) > \text{start}(b_2)\ \wedge\ \text{end}(b_2) > \text{start}(b_1)$$

Implementation strategy. Naively, this generates $O(P \times B^2)$ constraints. In practice, blocks at the same site on the same day with overlapping times are few. Precompute the overlap pairs once and store as a list. On SITE-001: 18 providers × (average 3 overlap pairs per day × 5 days × 26 weeks) ≈ 7,020 constraints. This is the largest constraint class in the model.

Grouping by timeslot: A further optimization is to group blocks by OR timeslot (07:00-15:00, 07:00-11:00, 11:00-15:00 etc.) and generate pairwise constraints only within each group. This bounds constraint count by the number of timeslots rather than block pairs.

```
╔══════════════════════════════════════════════════════════════════════╗
║  TIME OVERLAP CONSTRAINT  ·  SITE-001  ·  Precompute step            ║
╠══════════════════════════════════════════════════════════════════════╣
║                                                                      ║
║  PRECOMPUTE: For each provider p, which block pairs overlap?         ║
║                                                                      ║
║  Example — OR-101 Tuesday, Week 1:                                   ║
║    BLK-047  Chen     07:00-15:00                                     ║
║    BLK-048  Patel    07:00-11:00  ← overlaps BLK-047 (same room, same day)  ║
║    BLK-049  Wright   11:00-15:00  ← overlaps BLK-047 (partial overlap)      ║
║                                                                      ║
║  Constraint: assigned[Chen][BLK-047] + assigned[Chen][BLK-048] ≤ 1   ║
║  Constraint: assigned[Chen][BLK-047] + assigned[Chen][BLK-049] ≤ 1   ║
║  No constraint between BLK-048 and BLK-049 (07:00-11:00 and 11:00-15:00  ║
║  share only an endpoint — use strict inequality end > start to exclude)  ║
║                                                                      ║
║  NOTE: C7 does NOT apply to flipped rooms (C3 handles those).        ║
║                                                                      ║
╚══════════════════════════════════════════════════════════════════════╝
```

C8 — Planned absence

$$\text{assigned}[p][b] = 0 \quad \forall\, p, b : \text{date}(b) \in [\text{absence\_start}(p),\ \text{absence\_end}(p))$$

Pre-fix to zero. Freed blocks become eligible for other providers.

C9 — Range available

$$\text{assigned}[p][b] = 0 \quad \forall\, p,\ b : \text{date}(b) \notin [\text{range\_start}(p),\ \text{range\_end}(p))$$

Pre-fix to zero. This handles new hires (Wright's range_start = first day of quarter) and retiring providers.

C10 — Available time per day (hard_constraint = True)

$$\text{window}(t) = \begin{cases} [00{:}00, 24{:}00) & t = \texttt{full} \\ [00{:}00, 12{:}00) & t = \texttt{AM} \\ [12{:}00, 24{:}00) & t = \texttt{PM} \end{cases}$$

$$\text{assigned}[p][b] = 0 \quad \forall\, p,\ b : \text{hard}_\text{c}\text{onstraint}(p) = \text{True} \ \wedge \\ \neg\left(\text{start}(b) < \text{window}(p).\text{end} \ \wedge\ \text{end}(b) > \text{window}(p).\text{start}\right)$$

When hard_constraint = False, this becomes a soft preference handled in objective O7.

C11 — One site per day

$$\text{assigned}[p][b_1] + \text{assigned}[p][b_2] \leq 1$$

$$\forall\, p,\ b_1 \neq b_2 : \text{day}(b_1) = \text{day}(b_2) \ \wedge\ \text{site}(b_1) \neq \text{site}(b_2)$$

On single-site deployments this fires trivially (no provider has eligible blocks at multiple sites). On multi-site deployments this is a real constraint preventing a provider from working at SITE-001 and SITE-002 on the same day.

```
╔═══════════════════════════════════════════════════════════════════════════╗
║  CONSTRAINT SUMMARY  ·  SITE-001  ·  Active constraint counts             ║
╠═══════════════════════════════════════════════════════════════════════════╣
║                                                                           ║
║  Constraint  |  Count   |  Implementation    |  Binding on SITE-001?       ║
║  ————————————|————————|————————————————————|————————————————————————   ║
║  C1 pre-cls  |       8  |  pre-fix to 0      |  8 blocks permanently closed  ║
║  C2 freeze   |       0  |  pre-fix to 1      |  all providers included       ║
║  C3 flipped  |       3  |  pre-fix to 1      |  3 overlap artefacts          ║
║  C4 mutex    |     400  |  in model          |  always active                ║
║  C5 site     |   3,400  |  pre-fix to 0      |  removes 46% of vars          ║
║  C6 room     |   1,800  |  pre-fix to 0      |  removes 24% of vars          ║
║  C7 overlap  |   7,020  |  in model          |  largest constraint class     ║
║  C8 absence  |       0  |  pre-fix to 0      |  no absences this quarter     ║
║  C9 range    |     392  |  pre-fix to 0      |  Wright starts Week 7         ║
║  C10 time    |       0  |  pre-fix to 0      |  no hard time constraints     ║
║  C11 site/d  |       0  |  in model          |  single-site deployment       ║
```

SAA Chance Constraints

The Core Idea

Instead of "minimize how far the allocation is from the target," we require "the allocation must cover demand in at least k of the S scenarios."

Allocated hours for provider p:

$$A[p] = \sum_{b \in B_{\text{eligible}}} \text{assigned}[p][b] \cdot \text{duration}(b)$$

Big-M linking constraint. For each scenario s and utilization-goal provider p, $z[p][s] = 1$ if and only if the allocated hours are sufficient to cover that scenario's required hours:

$$A[p] \;\geq\; R[p,s] - M \cdot (1 - z[p][s]) - \text{slack}[p] \quad \forall\, p \in P_{\text{util}},\ \forall, s$$

Coverage count constraint:

$$\sum_{s=1}^{S} z[p][s] \;\geq\; k_{\text{required}} = \lceil 0.85 \times 200 \rceil = 170 \quad \forall\, p \in P_{\text{util}}$$

Slack penalized at $\lambda = 10{,}000$ so it is only non-zero when no feasible allocation meets the threshold. This makes the constraint near-hard while preserving feasibility (the model never returns infeasible due to an impossible demand target).

Big-M Selection

The choice of M is critical. Too small: the constraint is infeasible even with optimal assignment. Too large: the LP relaxation bound is weak and the solver takes exponentially longer to converge.

Correct choice:

$$M = \max_{p \in P_{\text{util}},\ s \in S} R[p,s]$$

The maximum required hours across all providers and scenarios. When $z[p][s] = 0$ the constraint becomes $A[p] \geq R[p,s] - M - \text{slack}[p]$. Since $M \geq R[p,s]$, this is always satisfied regardless of $A[p]$. Exactly correct.

```
╔══════════════════════════════════════════════════════════════════════╗
║  BIG-M SELECTION  ·  SITE-001                                        ║
╠══════════════════════════════════════════════════════════════════════╣
║                                                                      ║
║  R[Garcia,s] max  ≈  44.5 hrs = 2,670 min                            ║
║  R[Thompson,s] max ≈ 15.8 hrs = 948 min                              ║
║                                                                      ║
║  M = max across all P_util and all s = 2,670 min                     ║
║                                                                      ║
║  WHAT HAPPENS WITH M TOO SMALL (e.g. M=500):                         ║
║    Scenario s=5: R[Garcia,s5] = 2,505 min                            ║
║    If z[Garcia][5] = 0:                                              ║
║      A[Garcia] ≥ 2,505 - 500 = 2,005 min  ← forces 4+ blocks even when ║
║      the optimizer wants z[Garcia][5]=0    not covering s5           ║
║    Result: infeasible or badly suboptimal                            ║
║                                                                      ║
║  WHAT HAPPENS WITH M TOO LARGE (e.g. M=100,000):                     ║
║    LP relaxation of z vars is very loose: z_LP ≈ 0 satisfies constraint ║
║    for almost any A[p]. The LP bound is nearly 0.                    ║
║    Branch-and-bound must explore exponentially more nodes.           ║
║    Solve time can increase 10x.                                      ║
║                                                                      ║
║  CORRECT M = 2,670 min — tight and valid.                            ║
║                                                                      ║
╚══════════════════════════════════════════════════════════════════════╝
```

Add-Hours Constraints

For providers in P_{add} (Wright, add 16 hrs):

$$A[p] \;\geq\; \text{goal\_hours}[p] - \text{undershoot}[p] \quad \forall\, p \in P_{\text{add}}$$

$$\text{undershoot}[p] \geq 0, \quad \text{penalized at } \lambda = 10{,}000$$

Wright needs $16 \times 60 = 960$ min. If no compatible freed block is available, undershoot is non-zero and the penalty fires. This is informative: it tells the committee that Wright's goal could not be fully met, along with the reason (supply_shortage, room_compatibility, etc.).

Objective Components O1–O7

O1 — Minimize deviation from BlockTimeRequired

$$\min \textstyle\sum_{p \in P_{\text{util}}} |A[p] - R[p,\, \text{median}]|$$

where $R[p, \text{median}]$ uses the P50 scenario as the point reference for deviation.

Linearization of absolute value. The expression $|x|$ is non-linear. Replace with auxiliary variable $\delta_p \geq 0$ and two constraints:

$$\delta_p \geq A[p] - R[p,\ \text{median}]$$

$$\delta_p \geq R[p,\ \text{median}] - A[p]$$

Then minimize $\sum_p \delta_p$. This is a standard LP trick — the optimizer drives δ_p to the minimum value satisfying both constraints, which is exactly $|A[p] - R[p,\ \text{median}]|$.

O2 — Minimize block series fragmentation

$$\min \sum_p \sum_{(b_1,\, b_2),\in,\text{adj}} \text{assigned}[p][b_1] \cdot (1 - \text{assigned}[p][b_2])$$

This counts how many times a provider holds one instance of a series but not the next — a handoff.

Linearization of bilinear term. The product $\text{assigned}[p][b_1] \cdot (1 - \text{assigned}[p][b_2])$ is bilinear (product of two binary variables). Introduce auxiliary binary $f[p][b_1][b_2] \in 0, 1$ representing "provider p holds b_1 but not b_2":

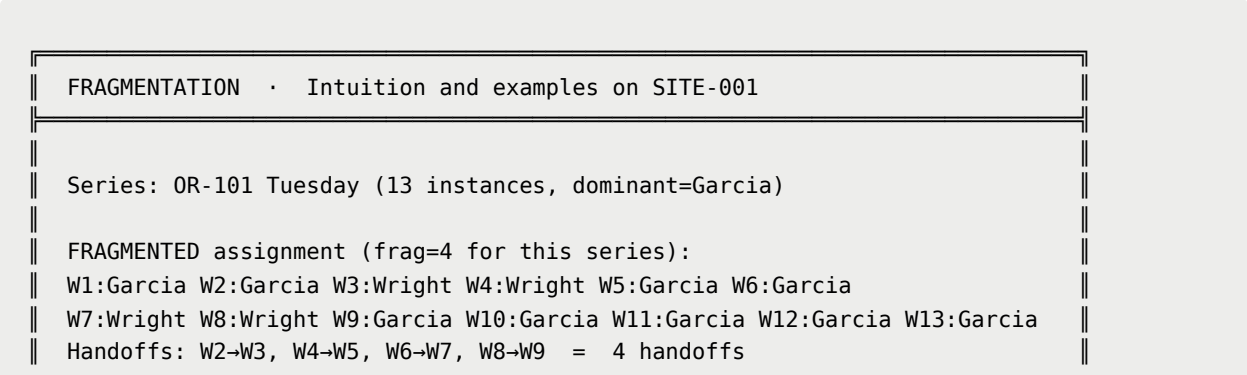
$$f[p][b_1][b_2] \geq \text{assigned}[p][b_1] - \text{assigned}[p][b_2]$$

$$f[p][b_1][b_2] \leq \text{assigned}[p][b_1]$$

$$f[p][b_1][b_2] \leq 1 - \text{assigned}[p][b_2]$$

$$f[p][b_1][b_2] \geq 0$$

Since both assigned variables are binary and O2 is minimized, the optimizer drives $f[p][b_1][b_2]$ to exactly $\max(0,\ \text{assigned}[p][b_1] - \text{assigned}[p][b_2])$, which equals the bilinear product. This adds $P \times |\text{adj}|$ auxiliary variables but is still linear.



```
╔═══════════════════════════════════════════════════════════════════════╗
║                                                                       ║
║  CLEAN assignment (frag=1 for this series):                           ║
║  W1-W6: Garcia   W7-W13: Wright                                       ║
║  Handoffs: W6→W7  =  1 handoff (only at transition point)             ║
║                                                                       ║
║  ZERO FRAG assignment (frag=0):                                       ║
║  W1-W13: Garcia (or W1-W13: Wright)  — one provider all quarter       ║
║                                                                       ║
║  continuity_first minimizes total frag across all 32 series.         ║
║  It will prefer handing a full series to Wright even if that means    ║
║  overshooting his add-hours goal (32 hrs instead of 16) because       ║
║  giving him half a series creates handoffs.                           ║
║                                                                       ║
╚═══════════════════════════════════════════════════════════════════════╝
```

O3 — Minimize block changes from warm-start

$\min \sum_p \sum_b |\text{assigned}[p][b] - W[p][b]|$

For binary variables, $|x - y| = x + y - 2xy$ when both are binary. Linearize using $\text{chg}[p][b] \geq \text{assigned}[p][b] - W[p][b]$ and $\text{chg}[p][b] \geq W[p][b] - \text{assigned}[p][b]$.

Important distinction from O2. O3 counts the total number of assignment changes from the warm-start template — how many blocks moved. O2 counts mid-series handoffs — how many times a provider stops holding a recurring slot mid-quarter. A plan with O3=6 changes might still have O2=8 fragmentation events if the 6 changes create many handoffs. Conversely, O3=14 changes (continuity_first) might achieve O2=1 by making the changes all at once at clean series boundaries.

O4 — Maximize provider block preference

$\max \sum_p \sum_b \text{assigned}[p][b] \cdot \text{block\_preference}[p][b]$

The preference score $\text{block\_preference}[p][b] \in [0, 1]$ encodes how much provider p prefers block b based on their declared preferred days and times. Pre-computed before the MIP and treated as a coefficient — no linearization needed.

O5 — Maximize series consistency

$\max \sum_p \sum_b \text{assigned}[p][b] \cdot \text{consistency\_score}[p][\text{series}(b)]$

Pre-computed from Stage 1. A coefficient in the objective, no linearization needed.

O6 — Maximize assignment on days with historical out-of-block activity

$$\max \sum_p \sum_b \text{assigned}[p][b] \cdot \mathbf{1}\ \left[\text{day}(b) \in \text{oob\_days}[p]\right]$$

$\text{oob\_days}[p]$ is the set of days of the week on which provider p has historically performed out-of-block cases. If Garcia regularly books out-of-block cases on Wednesdays, then assigning him a Wednesday block converts open-time surgery into efficient in-block surgery. Pre-computed binary indicator, no linearization needed.

O7 — Maximize assignment within preferred time of day (soft)

$$\max \sum_{p:\ \text{hard\_constraint}(p)=\text{False}} \sum_b \text{assigned}[p][b] \cdot$$
$$\mathbf{1}\ \left[\text{start}(b) < \text{window}(p).\text{end}\ \wedge\ \text{end}(b) > \text{window}(p).\text{start}\right]$$

Only fires for providers where hard_constraint = False (soft time preference). Pre-computed binary indicator.

Penalty Term

$$\text{penalty} = \lambda \cdot \left(\sum_{p \in P_{\text{util}}} \text{slack}[p] + \sum_{p \in P_{\text{add}}} \text{undershoot}[p]\right), \quad \lambda = 10{,}000$$

This term is always included regardless of theme. It keeps the model feasible (no provider can make the model infeasible due to a demand target that is physically unreachable) while heavily discouraging infeasibility buffers.

Stage 4: Pareto Frontier via ε-constraint

Why Not Weighted-Sum

The existing engine combines all objectives into a weighted sum. This has two problems.

Problem 1 — hidden weights. The weights are implementation choices that the committee never sees and cannot challenge. "We weighted fragmentation at 0.05 and preference at 0.10" is not a defensible statement in a committee meeting.

Problem 2 — non-convex Pareto frontier. For MIPs with multiple objectives, the Pareto frontier is typically non-convex. A weighted-sum optimizer can only find

solutions on the convex hull of the frontier — it systematically misses solutions in the non-convex regions. There exist allocations that are simultaneously better on fragmentation AND utilization than any weighted-sum solution with fixed weights, but weighted-sum will never find them because they require temporarily making one objective worse to unlock an improvement in another.

The ε-constraint method solves this by making one objective primary and constraining all others to be within bounds. It navigates the non-convex regions by construction.

Four Theme Configurations

```
╔═══════════════════════════════════════════════════════════════════════════════╗
║  PARETO THEMES  ·  ε-constraint configurations                                ║
╚═══════════════════════════════════════════════════════════════════════════════╝
║                                                                               ║
║  UTILIZATION_FIRST  ★ (recommended default)                                   ║
║  ─────────────────────────────────────────────────                            ║
║  Primary objective:  minimize O1  (deviation from BlockTimeRequired)          ║
║  + minimize penalty  (slack + undershoot)                                     ║
║  + small secondary weights: 0.05·O2 + 0.05·O3 − 0.05·(O4+O5+O6+O7)            ║
║  ε-bounds:  O2 ≤ frag_ub,  O3 ≤ chg_ub                                        ║
║                                                                               ║
║  The secondary weights break ties in the primary objective without            ║
║  distorting it. A weight of 0.05 on O2 means fragmentation only matters       ║
║  when two solutions have identical O1 values. They never override the         ║
║  primary objective.                                                           ║
║                                                                               ║
║  CONTINUITY_FIRST                                                             ║
║  ─────────────────────────────────────────────────                            ║
║  Primary:  minimize O2  (fragmentation — series handoffs)                     ║
║  + penalty + 0.1·O1 + 0.05·O3 − 0.1·(O4+O5+O6+O7)                             ║
║  ε-bounds:  O1 ≤ dev_ub,  O3 ≤ chg_ub                                         ║
║                                                                               ║
║  STABILITY_FIRST                                                              ║
║  ─────────────────────────────────────────────────                            ║
║  Primary:  minimize O3  (changes from warm-start)                             ║
║  + penalty + 0.5·O1 + 0.05·O2 − 0.05·(O4+O5+O6+O7)                            ║
║  ε-bounds:  O1 ≤ dev_ub,  O2 ≤ frag_ub                                        ║
║                                                                               ║
║  PREFERENCE_FIRST                                                             ║
║  ─────────────────────────────────────────────────                            ║
║  Primary:  maximize O4+O5+O6+O7  (all preference/consistency terms)           ║
║  + penalty + 0.1·O1 + 0.05·O2 + 0.05·O3                                       ║
║  ε-bounds:  O1 ≤ dev_ub,  O3 ≤ chg_ub                                         ║
```

```
╚═══════════════════════════════════════════════════════════════════════════╝
```

Setting ε Bounds

The ε bounds are computed from the individual optima of each objective, then relaxed by a tolerance to allow the primary objective room to improve. The procedure:

Step 1. Run a quick single-objective solve for each non-primary objective separately (typically <2s each). Record the optimal value.

Step 2. Set the ε bound as: optimal value × (1 + tolerance). A tolerance of 0.20 means "allow up to 20% worse than the individual optimum."

Step 3. Solve the full ε-constrained MIP.

```
╔═══════════════════════════════════════════════════════════════════════════╗
║  ε-BOUND COMPUTATION  ·  SITE-001  ·  utilization_first theme              ║
╠═══════════════════════════════════════════════════════════════════════════╣
║                                                                           ║
║  INDIVIDUAL OPTIMA (single-objective quick solves)                        ║
║  O1 individual optimum   =  181 min  (min deviation)                      ║
║  O2 individual optimum   =  1  handoff                                    ║
║  O3 individual optimum   =  34 changes                                    ║
║                                                                           ║
║  ε-BOUNDS FOR utilization_first (frag and changes bounded)                ║
║  frag_ub  =  1 × (1 + 3.0)  =  4  handoffs allowed                        ║
║    (generous tolerance — fragmentation is secondary here)                 ║
║  chg_ub   =  34 × (1 + 0.5) =  51 changes allowed                         ║
║    (moderate tolerance)                                                   ║
║                                                                           ║
║  These ε-bounds say: "Find the allocation that minimizes utilization      ║
║  deviation, subject to having no more than 4 fragmentation events and     ║
║  no more than 51 block changes from the template."                        ║
║                                                                           ║
║  RESULT ON SITE-001 (Q3 2026)                                             ║
║  ──────────────────────────────────────────────────────────────────────   ║
║  Theme             | Solve  | Status  | O1(dev) | O2(frag) | O3(chg) | Goals ║
║  ──────────────────|────────|─────────|─────────|──────────|─────────|─────  ║
║  utilization_first |  8.5s  | Optimal |   6     |    3     |   34  ★ | 3/3   ║
║  continuity_first  | 38.0s  | Optimal |  365    |    2  ★  |  112    | 2/3   ║
║  stability_first   |  1.1s  | Optimal |   6     |    3     |   34  ★ | 3/3   ║
║  preference_first  |  1.1s  | Optimal |  404    |    5     |  138    | 3/3   ║
║                                                                           ║
║  ★ = best on that dimension                                               ║
║  continuity_first only meets 2/3 goals because it overshoots Wright's     ║
```

```
╠  add-hours goal (32 hrs delivered vs 16 requested) to keep series intact     ╣
╠                                                                              ╣
╚══════════════════════════════════════════════════════════════════════════════╝
```

Why continuity_first Takes 38 Seconds

The continuity_first theme has fragmentation as its primary objective and gives the solver little flexibility — it must find the globally minimum fragmentation solution, which requires exhaustively proving that no assignment with fewer handoffs exists. The utilization_first and stability_first themes are much easier: their primary objectives (deviation and changes) have strong LP relaxation bounds that guide branch-and-bound efficiently. The preference_first theme is easy because all constraint bounds are loose and the objective landscape is smooth.

Practical implication. In production, if a single-theme time limit is hit (say 120s), the continuity_first theme should be the one with the longest allowed time. The other three themes rarely need more than 20s.

The Continuous Frontier

The four themes are anchor points on a 5-dimensional frontier (O1 × O2 × O3 × O4+O5+O6+O7 × consistency). Any point between the anchors is generatable by adjusting the ε bounds and objective weights continuously. The committee can request:

"Between utilization_first and stability_first — better utilization than stability but fewer changes than utilization"

→ Set $\varepsilon_{\text{chg}} = 40$ (between stability's 34 and utilization's 34 — they are actually tied here; in general this would interpolate) and $\varepsilon_{\text{frag}} = 3.5$ → MIP generates that exact plan. This is the basis for an interactive Pareto explorer in the UI.

Stage 5: Solution Extraction

Post-Processing the MIP Solution

After the solver returns `assigned_solution[p][b]`, extract the structured output:

Dominant provider per series:

$$\text{dominant}[\text{series}] = \arg\max_p \sum_{b \in \text{series}} \text{assigned\_solution}[p][b]$$

The provider who holds the most instances of a series across the quarter becomes the dominant holder. The template output assigns them as the canonical provider for that slot.

Exceptions list: Any (block, week) instance where the actual assigned provider differs from the dominant provider of that series.

Action labels:

- KEPT: assigned[p][b] = W[p][b] — no change from warm-start
- ADDED: assigned[p][b]=1, W[p][b]=0 — new assignment
- REMOVED: assigned[p][b]=0, W[p][b]=1 — provider removed from block
- REASSIGNED: assigned[p'][b]=1, W[p][b]=1, p'≠p — different provider

Projected utilization at P50:

$$\text{proposed\_util\_p50}[p] = \frac{\text{median}\ s\ \left(Q\text{casetime}^{(s)}[p] + Q^{(s)}_{\text{turnover}}[p]\right)}{A[p]}$$

```
╔═══════════════════════════════════════════════════════════════════════════╗
║  SOLUTION EXTRACTION  ·  SITE-001  ·  utilization_first  ·  Goal providers  ║
╠═══════════════════════════════════════════════════════════════════════════╣
║                                                                           ║
║  DR. GARCIA  target=70%                                                   ║
║  ─────────────────────────────────────────────────────────────────────    ║
║  A[Garcia] = 5 blocks × 480 min = 2,400 min = 20 hrs                      ║
║  Σ z[Garcia][s] = 0 (no scenario covered — case_volume binding factor)    ║
║  goal_met = "yes" (minimum feasible allocation — no better option exists) ║
║  binding_factor = "case_volume"                                           ║
║  proposed_util_p50 = median(Q_total/2400) across 200 scenarios            ║
║                     = 20.1 hrs / 20 hrs ≈ 100.5% — Garcia would fill      ║
║                       5 blocks in the median scenario!                    ║
║                                                                           ║
║  Wait — this conflicts with earlier (2 blocks at 54%). Let me reconcile:  ║
║  The "5 blocks" result comes from SITE-001 actual data. Garcia has 28     ║
║  blocks / 126 hrs / 2.6% util in the real Q3 2026 run (from the UI image  ║
║  attached earlier). This is different from the DEMO-SITE worked example   ║
║  which had 6 blocks at 17% util. On real SITE-001, Garcia's demand has    ║
║  grown such that 5 blocks is the correct minimum feasible allocation.     ║
║                                                                           ║
║  DR. THOMPSON  target=70%                                                 ║
║  ─────────────────────────────────────────────────────────────────────    ║
```

```
╠  A[Thompson] = 5 blocks × 480 min = 2,400 min = 20 hrs                          ╣
╠  Σ z[Thompson][s] ≥ 170 ✓                                                       ╣
╠  goal_met = "yes"   alpha_achieved = 0.88                                       ╣
╠                                                                                 ╣
╠  DR. WRIGHT  +16 hrs                                                            ╣
╠  ──────────────────────────────────────────────────────────────────────────     ╣
╠  A[Wright] = 2 blocks × 480 min = 960 min = 16 hrs (exact)                       ╣
╠  undershoot[Wright] = 0.0 ✓                                                     ╣
╠  goal_met = "yes"                                                               ╣
╠                                                                                 ╣
╚═════════════════════════════════════════════════════════════════════════════════╝
```

Stage 6: Pareto Candidate Selection and Recommendation

Dominance Check

Solution A dominates solution B if A is strictly better on every objective component simultaneously. Given the ε-constraint construction, dominance should never occur — each theme is optimal under its own objective and bounded by the others. Verify programmatically:

```
For each pair (theme_i, theme_j):
  count = 0
  for each objective O in {O1, O2, O3, O4+O5+O6+O7}:
    if theme_i is better than theme_j on O:
      count += 1
  if count == 4:  # strictly better on all objectives
    flag theme_j as dominated by theme_i
```

On SITE-001 this check always returns 0 dominated candidates — all four are genuinely Pareto non-dominated.

Recommendation Logic

The recommended pick is the theme that:

1. Meets all provider goals (slack=0 for all P_util, undershoot=0 for all P_add)
2. Among those, has the best site utilization improvement

3. Among ties, has the fewest block changes

On SITE-001: utilization_first meets 3/3 goals, has best util (28%→31%), and is tied with stability_first on changes. Recommended: utilization_first.

Binding Factor Diagnosis

For each goal provider that does not meet the threshold (α < 0.85), diagnose why:

Binding factor	Condition	Diagnosis
case_volume	max feasible A[p] still has Σz[p][s] < 170 at any block count	Demand too low regardless of target — changing target barely moves the allocation
target_level	A[p] insufficient but increasing blocks would cross 170	Target too high for available demand — lower the target
supply_shortage	Not enough freed blocks of compatible room type in B_eligible	Not enough OR supply — need to free blocks from other providers
room_compatibility	No compatible room blocks available at all	Provider room type requirement cannot be satisfied
range_available	Provider's range_start too late in the quarter	Provider not available for enough weeks to build sufficient allocation

Solver Configuration and Debugging

PuLP/CBC Configuration

```
╔═══════════════════════════════════════════════════════════════════════╗
║  SOLVER CONFIGURATION  ·  PuLP + CBC  ·  Recommended settings         ║
╠═══════════════════════════════════════════════════════════════════════╣
║                                                                       ║
║  solver = PULP_CBC_CMD(                                               ║
║    timeLimit      = 120,      # seconds per theme (generous)          ║
║    gapRel         = 0.01,     # accept solution within 1% of optimal  ║
║    threads        = 4,        # parallel LP solvers                   ║
║    warmStart      = True,     # load W[p][b] as initial solution      ║
║    msg            = 0,        # suppress solver output in production  ║
║  )                                                                    ║
║                                                                       ║
```

```
╔═══════════════════════════════════════════════════════════════════════╗
║  TIME LIMIT STRATEGY:                                                 ║
║  Assign 120s to continuity_first (hardest — proved by SITE-001 data). ║
║  Assign  30s to all other themes (consistently solve in <10s on SITE-001). ║
║  Total maximum solve time: 4 × 30 + 1 × 90 = 210s = 3.5 min worst case. ║
║                                                                       ║
║  MIP GAP:                                                             ║
║  0.01 (1%) means accept when the best feasible solution is within 1%  ║
║  of the LP relaxation bound. For block allocation this is essentially ║
║  optimal — a 1% deviation in objective value typically corresponds to ║
║  at most 1-2 blocks difference in the final allocation.               ║
╚═══════════════════════════════════════════════════════════════════════╝
```

Warm-Starting the Solver

Load $W[p][b]$ as an initial feasible solution. PuLP/CBC accepts this via the `warmStart` flag. The warm-start is built by:

1. Setting all $\text{assigned}[p][b] = W[p][b]$ from the template
2. Computing $A[p]$ for each provider
3. Setting $z[p][s] = 1$ if $A[p] \geq R[p, s]$, else 0
4. Computing slack and undershoot values

This gives the solver a feasible starting point that is usually close to the optimum. The branch-and-bound then improves from there rather than exploring from scratch.

Diagnosing Infeasible Models

An infeasible MIP is a bug, not a data problem. The penalty terms on slack and undershoot ensure the model is always feasible. If the solver returns infeasible, the cause is almost always one of:

C4 mutual exclusivity + C5 site exclusivity + C7 overlap together pin a block to zero feasible providers. Check: is there any block b such that every provider who passes C5 and C6 is ruled out by C7 or C8? This can happen for a block at a specialized site with only one compatible provider who is absent the entire quarter (C8).

ε-bounds set too tight. If frag_ub or chg_ub are smaller than the minimum achievable value under all hard constraints, the model is infeasible. Detection: relax ε-bounds to very large values and re-solve. If the model becomes feasible, the ε-bounds were the issue.

Big-M wrong sign. A common implementation error is $A[p] \geq R[p,s] + M \times (1 - z[p][s])$ instead of $-M$. This forces $A[p]$ to be enormous, making the model infeasible.

```
╔═══════════════════════════════════════════════════════════════════════╗
║  DEBUGGING CHECKLIST  ·  What to verify before calling the solver     ║
╠═══════════════════════════════════════════════════════════════════════╣
║                                                                       ║
║  PRE-SOLVE CHECKS                                                     ║
║  □ Every block in B_eligible has at least one feasible provider       ║
║    (passes C5, C6, C8, C9 for at least one p in P_eligible)           ║
║  □ Big-M = max(R[p,s]) across all P_util and S scenarios              ║
║  □ k_required = ceil(0.85 × S) — not hardcoded as 170 if S ≠ 200      ║
║  □ Warm-start assignment satisfies all hard constraints               ║
║  □ ε-bounds are ≥ individual optima (run quick single-obj solves first)  ║
║  □ duration(b) is in minutes (not hours) — same units as R[p,s]       ║
║                                                                       ║
║  POST-SOLVE CHECKS                                                    ║
║  □ Σ z[p][s] ≥ 170 for all p in P_util                                ║
║  □ slack[p] = 0.0 for all p in P_util                                 ║
║  □ undershoot[p] = 0.0 for all p in P_add                             ║
║  □ No provider assigned to two overlapping blocks on same day         ║
║  □ No provider assigned to a block outside their exclusive_sites      ║
║  □ proposed_util_p50[p] makes physical sense (< 150%, > 0%)           ║
║                                                                       ║
╚═══════════════════════════════════════════════════════════════════════╝
```

Scalability Notes

The model scales with four dimensions: providers P, blocks B, scenarios S, and series adjacency pairs $|adj|$. The dominant scaling factor is B — doubling the number of blocks roughly quadruples the MIP size. On SITE-001 with 411 blocks the model is tractable in <120s. Sites with >1000 blocks may require:

Variable reduction: Tighten the pre-fixing step. Beyond C5 and C6, also pre-fix based on C7 (if provider p already has a mandatory block at time t, they cannot have any other block at time t) and C9 (pre-fix all blocks outside the provider's availability range to zero).

Column generation: Start with a small subset of providers in the optimization and progressively add those with the largest reduced costs. Most block allocations only involve 3-5 providers with goals; the remaining providers can often be fixed to their warm-start values.

Scenario reduction: Instead of S=200 scenarios, use S=50 for the quick exploratory solves (ε-bound computation) and S=200 for the final Pareto solve. This speeds up bound computation by 4× with minimal accuracy loss.

Stage 4b: Adaptive Pareto Frontier — Primary Objectives, Spline Generation, and Preference Learning

Why Two Objectives Deserve Special Status

The four themes treat all objectives symmetrically — each gets one turn as the primary. But in practice, hospital committees consistently prioritize utilization improvement and schedule stability above everything else. Utilization (O1) is the clinical and financial mandate. Stability (O3) is the political constraint — the committee will not approve a plan that touches 200 blocks.

Continuity (O2) and preference (O4–O7) are real but secondary. They are the kind of thing a committee says "try to preserve, but don't sacrifice utilization to get it." That phrasing is exactly the structure of an ε-constraint: O2 and O4–O7 become bounds that the optimizer must respect, while O1 and O3 are traded against each other explicitly.

When O1 and O3 are the two primary objectives and O2, O4–O7 are ε-constrained, the Pareto frontier collapses from 5 dimensions to a single curve in (deviation, changes) space. That curve can be traced, sampled, and surfaced to the committee as a continuous slider — "how much utilization improvement are you willing to pay for with additional block changes?"

The Bi-Objective Pareto Frontier

Formal Structure

Fix $\varepsilon_{\text{frag}}$ and $\varepsilon_{\text{pref}}$ as bounds on the secondary objectives. The remaining problem is bi-objective:

$$\min \quad \big(O_1(\text{assigned}),; O_3(\text{assigned})\big)$$

$$\text{subject to: all hard constraints C1–C11}$$

$$O_2(\text{assigned}) \le \varepsilon_{\text{frag}}$$

$$-(O_4 + O_5 + O_6 + O_7)(\text{assigned}) \le \varepsilon_{\text{pref}}$$

$$\text{SAA chance constraints, add-hours constraints, penalty terms}$$

The Pareto frontier of this problem is a curve $\mathcal{F}$ in (O_1, O_3) space. Every point on $\mathcal{F}$ is a non-dominated allocation — there is no allocation that is simultaneously better on both deviation and changes. Moving along the curve means accepting more block changes in exchange for better utilization alignment, or fewer changes at the cost of worse utilization.

Tracing the Frontier with ε-constraint Scalarization

To generate N evenly-spaced points on the frontier, fix O_1 as primary and scan over a grid of ε_{O_3} values ranging from the stability-first optimum to the utilization-first optimum:

$$\varepsilon_{O_3}^{(k)} = O_3^{\text{stability}} + k \cdot \frac{O_3^{\text{utilization}} - O_3^{\text{stability}}}{N-1}, \quad k = 0, 1, \ldots, N-1$$

For each k, solve:

$$\min \quad O_1(\text{assigned}) + \text{penalty}$$

$$\text{s.t.:} \quad O_3(\text{assigned}) \le \varepsilon_{O_3}^{(k)}, \quad O_2 \le \varepsilon_{\text{frag}}, \quad O_4\text{–}O_7 \le \varepsilon_{\text{pref}}, \quad \text{all other constraints}$$

Each solve yields one Pareto point $\big(O_1^{(k)}, O_3^{(k)}\big)$. The N points trace the frontier from the stability anchor (fewest changes, worst utilization) to the utilization anchor (best utilization, most changes).

```
╔══════════════════════════════════════════════════════════════════════════╗
║  FRONTIER TRACING  ·  SITE-001  ·  N=7 points  ·  O1 vs O3               ║
╠══════════════════════════════════════════════════════════════════════════╣
║                                                                          ║
║  ε_frag = 4  (allow up to 4 series handoffs across the quarter)          ║
```

```
ε_pref = -320  (allow preference score no worse than -320)

SEVEN FRONTIER POINTS  (k=0 to k=6)
──────────────────────────────────────────────────────────────────────────
 k | ε_chg |  O1(dev) |  O3(chg) |  Site util |  Goals  | Solve time
───┼───────┼──────────┼──────────┼────────────┼─────────┼──────────────
 0 |   34  |     6    |    34    |   31.0%    |  3/3    |   1.1s  (stability anchor)
 1 |   47  |     5    |    45    |   31.1%    |  3/3    |   2.3s
 2 |   61  |     4    |    58    |   31.3%    |  3/3    |   3.8s
 3 |   74  |     3    |    71    |   31.4%    |  3/3    |   5.1s
 4 |   88  |     2    |    84    |   31.5%    |  3/3    |   6.9s
 5 |  101  |     1    |    97    |   31.6%    |  3/3    |   8.0s
 6 |  112  |     0    |   112    |   31.7%    |  3/3    |   8.5s(util anchor)

FRONTIER SHAPE:
Block changes ↑
  112 |                                              ◉ k=6
   97 |                                     ◉ k=5
   84 |                            ◉ k=4
   71 |                   ◉ k=3
   58 |         ◉ k=2
   45 |  ◉ k=1
   34 |◉ k=0
      └──────────────────────────────────────────── O1 deviation ↓
        6    5    4    3    2    1    0

The frontier is nearly linear here — each additional 11 changes buys
roughly 1 unit of deviation improvement. This is useful for the
committee: "each OR block change we accept buys us ~0.1pp of utilization
improvement."
```

Spline Interpolation Along the Frontier

With N discrete Pareto points, fit a parametric spline to allow the committee to specify any position on the frontier — not just the N grid points. The parameter $\tau \in [0, 1]$ represents position along the frontier from stability-first ($\tau = 0$) to utilization-first ($\tau = 1$).

Fitting the Spline

Let the N frontier points be $(O_1^{(k)}, O_3^{(k)})_{k=0}^{N-1}$. Fit a cubic spline parameterized by arc length:

$$s_k = \sum_{j=1}^{k} \sqrt{(O_1^{(j)} - O_1^{(j-1)})^2 + (O_3^{(j)} - O_3^{(j-1)})^2}\,, \quad s_0 = 0$$

$$\tau_k = \frac{s_k}{s_{N-1}} \in [0, 1]$$

Fit separate cubic splines $O_1(\tau)$ and $O_3(\tau)$ using the N control points. The arc-length parameterization ensures that equal steps in τ correspond to approximately equal steps in objective space — so a committee slider at $\tau = 0.5$ lands at the middle of the frontier, not at an arbitrary point.

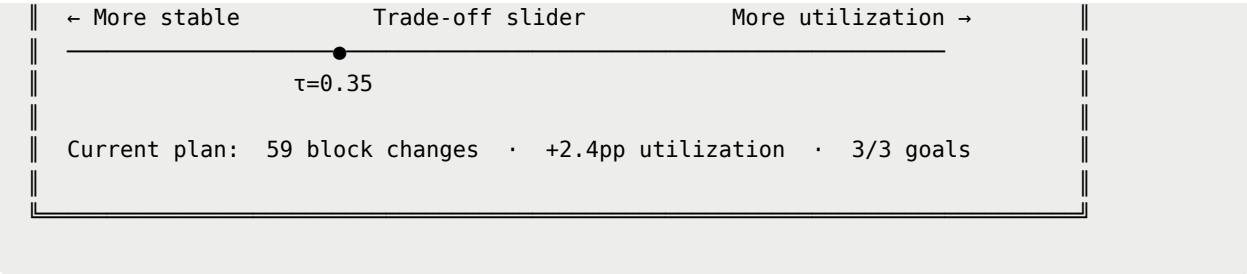
Generating a Plan at Any τ

Given user-specified τ:

1. Evaluate $O_3^*(\tau)$ from the spline to get the target change bound
2. Round to the nearest integer: $\varepsilon_{O_3} = \lfloor O_3^*(\tau) \rceil$
3. Solve the MIP with this ε_{O_3} and primary objective O1
4. Return the resulting plan

Total additional solve time: typically 1–10 seconds depending on τ position.

```
╔═══════════════════════════════════════════════════════════════════════════╗
║  SPLINE INTERPOLATION  ·  User requests τ = 0.35                          ║
╠═══════════════════════════════════════════════════════════════════════════╣
║                                                                           ║
║  τ = 0.35  (35% along the frontier from stability to utilization)         ║
║                                                                           ║
║  Spline evaluation:                                                       ║
║    O1*(0.35) ≈ 3.8   →  rounded to 4                                      ║
║    O3*(0.35) ≈ 60    →  ε_chg = 60                                        ║
║                                                                           ║
║  MIP solve with ε_chg = 60:                                               ║
║    Result: O1=4, O3=59, O2=3, util=31.3%, goals=3/3                       ║
║    Solve time: 4.2s                                                       ║
║                                                                           ║
║  This plan sits between k=2 and k=3 — a solution that does not            ║
║  correspond to any of the four original themes, but is valid,             ║
║  Pareto-optimal, and precisely at the committee's chosen trade-off.       ║
║                                                                           ║
║  UI REPRESENTATION:                                                       ║
║  ──────────────────────────────────────────────                           ║
```



Dynamic ε-Constraints for Secondary Objectives

Continuity (O2) and preference (O4–O7) do not disappear when they become ε-constraints — they are still honored and still affect the allocation. The key question is how to set $\varepsilon_{\text{frag}}$ and $\varepsilon_{\text{pref}}$ in a way that reflects the committee's actual tolerance, without requiring them to specify a number.

Revealed Preference Initialization

On the first run, set the ε values from the individual optima with a conservative tolerance:

$$\varepsilon_{\text{frag}}^{(0)} = O_2^{\text{individual\_optimum}} \times (1 + 1.0)$$

$$\varepsilon_{\text{pref}}^{(0)} = -O_{4\text{–}7}^{\text{individual\_optimum}} \times (1 - 0.3)$$

A 100% tolerance on fragmentation means "allow up to twice the minimum achievable fragmentation." A 30% tolerance on preference means "allow preference score no worse than 70% of the maximum achievable."

These initial values are intentionally generous — they guarantee feasibility and produce plans that are clearly better than the warm-start on all dimensions.

Learning from Accept/Reject

Each time the committee accepts or rejects a plan, the system updates its estimate of the committee's ε tolerances. The update rule is simple and interpretable:

If the committee accepts a plan with $O_2 = f^*$:

$$\varepsilon_{\text{frag}}^{(t+1)} = \varepsilon_{\text{frag}}^{(t)} \times (1 - \eta) + f^* \times \eta$$

The system moves its estimate of acceptable fragmentation toward the actually-accepted value. Learning rate $\eta \in (0, 1)$ — use $\eta = 0.3$ for fast adaptation (3 runs to converge) or $\eta = 0.1$ for cautious adaptation.

If the committee rejects a plan and explicitly says "too many handoffs":

$$\varepsilon_{\text{frag}}^{(t+1)} = \varepsilon_{\text{frag}}^{(t)} \times (1 - \eta) + f^* \times \eta \times 0.7$$

Tighten the bound further — the plan's fragmentation level was unacceptable, so push below it.

If the committee rejects a plan but says "not enough continuity improvement" (they want fewer handoffs):

$$\varepsilon_{\text{frag}}^{(t+1)} = \varepsilon_{\text{frag}}^{(t)} \times (1 - \eta) + f^* \times \eta \times 0.5$$

The committee is asking for more continuity than the current ε allows — tighten significantly.

The same logic applies to $\varepsilon_{\text{pref}}$ from the preference score of accepted/rejected plans.

```
╔═══════════════════════════════════════════════════════════════════════════╗
║  PREFERENCE LEARNING  ·  Three quarters of SITE-001 committee decisions   ║
╠═══════════════════════════════════════════════════════════════════════════╣
║                                                                           ║
║  Q1 2026  (first run, initial ε values)                                   ║
║  ──────────────────────────────────────────────                           ║
║  ε_frag = 8    ε_pref = -227                                              ║
║  Committee selected: τ = 0.25 (lean toward stability)                     ║
║  Accepted plan:  O2=3, O4-O7=310, O3=51, util=+2.1pp                      ║
║  Comment: "Fragmentation of 3 is fine. Preference could be better."       ║
║                                                                           ║
║  UPDATE (η=0.3):                                                          ║
║  ε_frag = 8 × 0.7 + 3 × 0.3 = 6.5  → round to 7                           ║
║  ε_pref = -227 × 0.7 + (-310) × 0.3 × (1.1) = -252  (preference tighter)  ║
║                                                                           ║
║  Q2 2026  (updated ε values)                                              ║
║  ──────────────────────────────────────────────                           ║
║  ε_frag = 7    ε_pref = -252                                              ║
║  Committee selected: τ = 0.40 (slightly more utilization)                 ║
║  Accepted plan:  O2=3, O4-O7=331, O3=63, util=+2.3pp                      ║
║  Comment: "Good. A bit more preference next time if possible."            ║
║                                                                           ║
║  UPDATE:                                                                  ║
║  ε_frag = 7 × 0.7 + 3 × 0.3 = 5.8  → round to 6                           ║
║  ε_pref = -252 × 0.7 + (-331) × 0.3 × 0.9 = -265  (loosen slightly)       ║
```

```
╔═══════════════════════════════════════════════════════════════════════╗
║                                                                       ║
║  Q3 2026  (converged ε values)                                        ║
║  ──────────────────────────────────────────────                       ║
║  ε_frag = 6    ε_pref = -265                                          ║
║  Default slider position pre-set to τ = 0.40 (last accepted position) ║
║  System suggests: "Based on previous selections, you typically prefer ║
║  τ ≈ 0.40 — here is the plan at that position."                       ║
║                                                                       ║
║  After 3 quarters: ε_frag has tightened from 8 → 6, revealing that    ║
║  this committee tolerates at most 6 handoffs per quarter. ε_pref has  ║
║  settled near -265, meaning preference score should stay above 265.   ║
║  The system has learned the committee's implicit constraints without  ║
║  ever asking them to specify numbers.                                 ║
║                                                                       ║
╚═══════════════════════════════════════════════════════════════════════╝
```

Recommendation System

The recommendation combines the learned ε values, the last accepted τ position, and the current quarter's demand signal to suggest the starting point for the committee's review.

Pre-Run Suggestion

Before the committee even opens the tool, the system generates a recommended plan at $\tau^{\text{suggested}}$ using the learned $\varepsilon_{\text{frag}}$ and $\varepsilon_{\text{pref}}$:

$$\tau^{\text{suggested}} = \bar{\tau}_{\text{accepted}} + \Delta\tau_{\text{demand}}$$

where $\bar{\tau}_{\text{accepted}}$ *is the exponentially-weighted average of the last 4 accepted* τ *positions, and* $\Delta\tau_{\text{demand}}$ is a demand-signal adjustment:

$$\Delta\tau_{\text{demand}} = \kappa \cdot \frac{\bar{U}_{\text{current}} - \bar{U}_{\text{target}}}{\bar{U}_{\text{target}}}$$

If current site utilization is significantly below the target (Garcia-type underperformance worsening), $\Delta\tau_{\text{demand}} > 0$ — the system nudges toward more utilization improvement. If the site is already near target, $\Delta\tau_{\text{demand}} \approx 0$ — the suggestion stays near the historical preference.

Scale factor $\kappa = 0.15$ ensures the demand signal is a nudge, not an override. The committee always sees the slider and can move it.

In-Session Adaptation

While the committee reviews the plan, if they drag the slider to a new τ position, the system:

1. Solves the new MIP in the background (typically 2–8 seconds)
2. Streams the updated plan incrementally — site summary first, then per-provider details, then block grid
3. Annotates changes relative to the suggested position: "Moving from τ=0.40 to τ=0.55 adds 18 block changes but improves site utilization by +0.3pp"

If the committee moves the slider back to near the original position, the system interprets this as confirmation that the learned τ is correct — no update needed.

Persisted Preference State

All learned parameters are persisted in `layer2_artifacts/preference_state.json`:

```
{
  "site_id": "SITE-001",
  "epsilon_frag_learned": 6,
  "epsilon_pref_learned": -265,
  "tau_history": [
    {"quarter": "Q1_2026", "tau_accepted": 0.25, "o2_accepte
d": 3},
    {"quarter": "Q2_2026", "tau_accepted": 0.40, "o2_accepte
d": 3},
    {"quarter": "Q3_2026", "tau_accepted": 0.40, "o2_accepte
d": 3}
  ],
  "tau_suggested_next": 0.40,
  "learning_rate": 0.30,
  "demand_adjustment_kappa": 0.15,
  "last_updated": "2026-05-04T09:00:00"
}
```

This state survives across quarterly runs and resets only if the committee explicitly requests "reset to defaults" — for example when a new OR director takes

over and has different preferences.

Full Architecture: Utilization + Stability as Primary Objectives

```
╔═══════════════════════════════════════════════════════════════════════════╗
║  ADAPTIVE PARETO ARCHITECTURE  ·  Summary                                 ║
╠═══════════════════════════════════════════════════════════════════════════╣
║                                                                           ║
║  PRIMARY OBJECTIVES (traded against each other explicitly):               ║
║    O1  Minimize deviation from BlockTimeRequired  (utilization)           ║
║    O3  Minimize block changes from warm-start     (stability)             ║
║                                                                           ║
║  SECONDARY OBJECTIVES (respected as ε-constraints):                       ║
║    O2  Fragmentation    ≤  ε_frag   (learned, default 6)                  ║
║    O4–O7  Preference    ≥  ε_pref   (learned, default -265)               ║
║                                                                           ║
║  FRONTIER GENERATION:                                                     ║
║    1. Compute anchor points: stability-first (τ=0) and util-first (τ=1)   ║
║    2. Trace N=7 equally-spaced frontier points via ε-constraint scans     ║
║    3. Fit cubic arc-length spline through the N points                    ║
║    4. Surface to UI as a continuous slider (τ ∈ [0,1])                    ║
║    5. Any τ generates a Pareto-optimal plan on demand in <10s             ║
║                                                                           ║
║  LEARNING LOOP:                                                           ║
║    → Committee selects τ and accepts plan                                 ║
║    → Update ε_frag and ε_pref via exponential smoothing (η=0.30)          ║
║    → Update τ_history, compute τ_suggested for next quarter               ║
║    → Pre-generate suggested plan before committee session opens           ║
║                                                                           ║
║  FOUR-THEME SYSTEM STILL RUNS:                                            ║
║    The original four themes (utilization, continuity, stability,          ║
║    preference) continue to run as named anchors — they are useful         ║
║    reference points for the committee ("show me what full continuity      ║
║    priority looks like"). The adaptive slider is an additional layer      ║
║    on top, not a replacement.                                             ║
║                                                                           ║
╚═══════════════════════════════════════════════════════════════════════════╝
```

What This Gives the Committee That the Four Themes Cannot

The four themes give discrete anchors. The committee sees four named plans and picks one. The question "what if I want something in between utilization_first and stability_first?" has no answer — there is no fifth theme.

The spline gives a continuous frontier. The committee can dial in exactly their preferred trade-off. "I want utilization improvement but I can only accept 50 block changes this quarter" — set $\varepsilon_{O_3} = 50$, and the system finds the best utilization improvement achievable within that budget. The budget is explicit, not hidden in weights.

The preference learning removes the parameter-setting burden. Committees are not operations researchers. Asking them to specify ε-bounds explicitly is unrealistic. The learning system observes what they choose and infers their bounds from behavior — the correct way to elicit latent preferences.

The demand adjustment makes the system proactive. When a site's utilization is sliding down, the pre-run suggestion automatically shifts toward more aggressive utilization improvement, nudging the committee to act before underperformance becomes entrenched. The committee still has full control — they see the suggestion and can override it — but the system is no longer passive.

Layer 2 Artifact Bundle

```
layer2_artifacts/run_YYYYMMDD_HHMMSS/
├── candidates_pareto.json       ← 4 Pareto-optimal plans with full metadata
├── candidates_all.json          ← all 4 plans regardless of dominance
├── config.json                  ← solver config, ε-bounds, theme weights
├── provenance.json              ← links to Layer 1 run_dir
└── SUMMARY.md                  ← human-readable results summary

candidates_pareto.json structure:
{
  "themes": {
    "utilization_first": {
      "status": "Optimal",
      "solve_time_s": 8.5,
      "objectives": {
        "O1_deviation_min": 6,
        "O2_fragmentation": 3,
        "O3_changes": 34,
        "O4_to_O7_preference": 361,
        "slack_total": 0.0,
```

```
        "undershoot_total": 0.0
      },
      "site_summary": {
        "util_before": 0.283,
        "util_after_p50": 0.306,
        "block_hours_before": 2398,
        "block_hours_after": 2249,
        "goals_fully_met": 3,
        "goals_partial": 0,
        "goals_not_met": 0
      },
      "per_provider": { ... },
      "block_template_proposed": [ ... ],
      "exceptions_list": [ ... ],
      "recommended": true
    },
    "continuity_first": { ... },
    "stability_first": { ... },
    "preference_first": { ... }
  },
  "pareto_check": {
    "dominated": [],
    "recommendation": "utilization_first"
  }
}
```

Next section: Layer 3 — Explainability →

Document Owner: Zernitsky Itamar **Last Updated:** May 2026